

AI-Powered URL Detector

Project Documentation, Detection Coverage & Workflow

Team Lead: A.V.V. Satyanarayana

Team Members: N. Sai Sudheer · K. Shanmukha

Project documentation

Table of Contents

- 1 1. Executive Summary 1
- 2 2. Project Objectives 1
- 3 3. Main Features 1
 - 3.1 3.1 URL Input and Scan Action 1
 - 3.2 3.2 Safe Result State 2
 - 3.3 3.3 Danger Result State 2
 - 3.4 3.4 Threat Library 2
- 4 4. Detection Coverage 2
- 5 5. End-to-End Workflow 3
 - 5.1 5.1 Workflow Diagram 4
- 6 6. Technical Architecture 4
- 7 7. Result Data Model 4
- 8 8. Testing and Verification 5
- 9 9. Security Limitations and Responsible Use 5
- 10 10. Future Enhancements 6
- 11 11. Team Details 6
- 12 12. Conclusion 6
- 13 References 7

1 1. Executive Summary

The **AI-Powered URL Detector** is a cyber-security web application that helps users inspect a URL before opening it. The user pastes a link into the scanner and selects **Analyze URL with AI**. The application evaluates URL structure, protocol, domain patterns, encoded content, redirect parameters, suspicious keywords, download indicators, and other attack signals. It then presents a clear result instead of a vague warning.

The result explains four important points: whether the URL appears safe or dangerous, the detected attack category, the likely purpose of the URL, and the possible impact a user should understand. The interface uses a dark cyber-security theme with cyan, emerald, indigo, and danger-red visual states so that the result is easy to interpret.

Important scope note. The current project contains a client-side pattern-based analysis engine for demonstration and academic project use. It does not replace a production threat-intelligence service, antivirus engine, browser isolation system, or security operations platform.

2 2. Project Objectives

The project is designed around a simple user need: a person receives a link and wants to know whether it is safe before clicking it. The application converts this need into an understandable security workflow.

Objective	Description
Fast inspection	Allow a user to paste a URL and receive a result within a short interaction.
Clear classification	Show an exact attack type such as PHISHING, SQL INJECTION VIA URL, or NO ATTACK DETECTED.
Security awareness	Explain why a URL is suspicious and what kind of harm may occur.
Accessible interface	Use clear labels, high contrast, keyboard-friendly controls, and safe/danger color states.
Extensible foundation	Keep the classifier organized so additional rules or real threat APIs can be connected later.

3 3. Main Features

The application contains a single focused scanner page. A compact header identifies the product and provides navigation to the workflow and threat library. The central hero section presents the purpose of the product and the scan card.

3.1 3.1 URL Input and Scan Action

The scan card contains a target URL field and an **Analyze URL with AI** button. The interface also provides safe and threat demonstration links so that the project can be tested without preparing sample inputs

manually. During analysis, the button changes to a running state and a progress indicator communicates that the URL is being inspected.

3.2 3.2 Safe Result State

When no obvious malicious pattern is detected, the application shows a safe-themed result. The result includes a confidence score, the label **URL appears safe**, the exact text **NO ATTACK DETECTED**, the likely purpose of the URL, and a caution that users should still verify a domain before entering sensitive information.

3.3 3.3 Danger Result State

When a suspicious pattern is detected, the application switches to a danger-themed result. The result includes a high-risk label, a confidence score, an exact attack category, a short explanation of the likely URL purpose, an educational attack explanation, and a signal-by-signal inspection list.

3.4 3.4 Threat Library

The threat library introduces common risks such as credential phishing, malware delivery, redirect chains, and tracking or impersonation. Its purpose is educational: a user can understand the general behavior of dangerous links even when no URL is currently being scanned.

4 4. Detection Coverage

The classifier supports the following fifteen requested categories. A category is selected when the URL contains matching indicators associated with that attack family. The application displays the category in uppercase as the **Attack type** result.

No.	Attack category	Typical indicators and user-facing explanation
1	Phishing URL Attack	Fake account alerts, verification requests, and login actions intended to steal sensitive information.
2	Malware URL Attack	Malware, trojan, ransomware, spyware, crack, keygen, or payload language and related file patterns.
3	Scam / Fraud URL Attack	Prize, refund, cashback, investment, job, giveaway, urgent payment, or “act now” themes.
4	Banking Phishing Attack	Bank, card, UPI, net banking, payment, OTP, or financial verification language.
5	Credential Harvesting Attack	Login, sign-in, password reset, account verification, OTP, wallet, or credential collection wording.
6	Typosquatting Attack	Brand-like domains with intentional spelling changes such as paypa1, g00gle, or micros0ft.
7	Homograph Attack	Lookalike Unicode characters or punycode domains beginning with xn–.
8	Open Redirect Attack	Redirect, next, continue, return, target, destination, or URL parameters that may hide a final page.

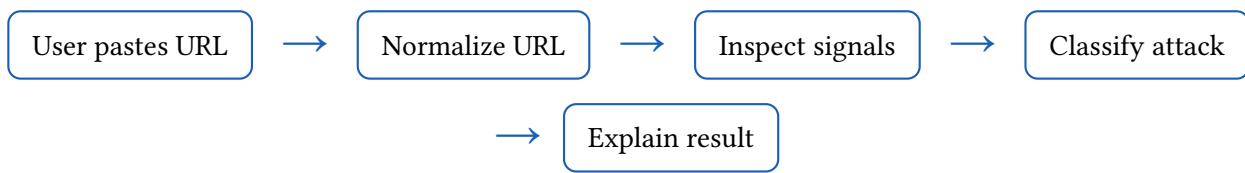
No.	Attack category	Typical indicators and user-facing explanation
9	Cross-Site Scripting (XSS) via URL	Script tags, javascript:, event handlers, encoded script syntax, alert(), or document.cookie patterns.
10	SQL Injection via URL	UNION SELECT, OR 1=1, SQL comments, information_schema, sleep, or waitfor delay patterns.
11	Malicious File Download Attack	Executable, installer, archive, APK, script, payload, or suspicious download paths.
12	Malicious Short URL Attack	Shorteners such as bit.ly, tinyurl, t.co, goo.gl, is.gd, ow.ly, and shorturl.at.
13	URL Obfuscation Attack	Encoded separators, nested encodings, Base64-like values, unusual ports, or user-info tricks.
14	Session / Token Theft via URL	Session IDs, PHPSESSID, JWT, auth_token, access_token, sid, or other credentials in parameters.
15	Drive-by Download Attack	Browser-update, codec-update, exploit-kit, silent-download, or automatic download indicators.

5.5. End-to-End Workflow

The workflow is intentionally linear so that a first-time user can understand what to do without security expertise.

1. **Paste:** The user copies a URL from an email, message, social-media post, QR code, or browser address bar and pastes it into the target field.
2. **Normalize:** The application trims whitespace and adds `https://` when a user enters a domain without a protocol.
3. **Inspect:** The classifier converts the URL to a normalized lowercase representation and checks protocol, host structure, query parameters, fragments, encoding, shorteners, suspicious top-level domains, and attack-specific signals.
4. **Match:** The attack rules are checked in priority order. The first matching category becomes the primary attack type. If no category matches but broad risk signals exist, the result becomes **SUSPICIOUS URL ATTACK**.
5. **Explain:** The application selects a purpose statement and educational attack explanation for the matched category.
6. **Present:** The UI renders either the safe state or the danger state with a score, exact attack type, explanation, and inspected signals.
7. **Educate:** The user can read the “Just for knowledge” section to understand what the detected attack can do and what behavior should be avoided.

5.1 Workflow Diagram



6 Technical Architecture

The current implementation is a static React web application. It is suitable for a demonstration, academic project, and user-interface prototype. The principal code is organized into a page component, global styles, application routing, and the static build configuration.

Layer	Implementation
Presentation layer	React 19 components rendered through Vite. The main experience is implemented in <code>client/src/pages/Home.tsx</code> .
Interaction layer	React state manages the entered URL, idle state, scanning state, safe result, danger result, and threat-library visibility.
Classification layer	The local <code>analyseUrl</code> function evaluates ordered attack rules and returns a typed result object.
Styling layer	Tailwind CSS and custom CSS provide the dark background, neon accents, cards, responsive layout, and reduced-motion support.
Icons and visual language	Lucide icons communicate scanning, links, shields, warnings, search, and workflow concepts.
Build layer	Vite bundles the frontend and TypeScript verifies the source before production output is generated.

7 Result Data Model

Each scan result contains the fields below. Keeping the result structure explicit makes the interface consistent across all attack categories.

Field	Purpose
status	Controls the safe or danger visual state.
score	Provides a confidence-style number for the current pattern analysis.
label	Provides the main result heading, such as High risk detected or URL appears safe.

Field	Purpose
attackType	Shows the exact category, such as PHISHING URL ATTACK or SQL INJECTION VIA URL.
summary	Provides the short reason for the current classification.
purpose	Explains what the URL is likely trying to do.
attack	Explains the possible impact for user awareness.
indicators	Lists individual checks such as domain reputation, transport security, URL structure, and known attack signals.

8 8. Testing and Verification

The project was verified through both static checks and live browser interaction. The following checks were completed after the fifteen-category classifier was added.

Verification	Outcome
TypeScript check	Passed with pnpm check.
Production build	Passed with pnpm build. The Vite frontend and server bundle were generated successfully.
Safe example	Displayed URL appears safe and NO ATTACK DETECTED.
Threat example	Displayed a danger result with an exact attack category.
XSS example	Displayed CROSS-SITE SCRIPTING (XSS) VIA URL with purpose and impact explanation.
Responsive preview	Checked the polished desktop layout and result-panel behavior in the live preview.

9 9. Security Limitations and Responsible Use

A URL pattern classifier cannot prove that a website is safe. A malicious actor can use a new domain, hide content behind redirects, serve different content to different visitors, or use a legitimate domain that has been compromised. Similarly, a safe-looking URL can still host unsafe content after the scan.

For production deployment, the application should be connected to trusted threat-intelligence services and should perform server-side checks. Recommended additions include DNS and certificate inspection, redirect resolution in a sandbox, domain-age and reputation checks, malware scanning, content isolation, rate limiting, logging, and secret management. The scanner should never fetch an untrusted URL directly in the user's browser merely to inspect it.

Users should avoid entering passwords, OTPs, banking details, recovery codes, or payment information into a suspicious page. They should independently open the official service through a known bookmark or manually typed domain when an important account action is requested.

10 10. Future Enhancements

The project has a clear path from an academic prototype to a production-oriented service.

Enhancement	Expected benefit
Threat-intelligence API	Add VirusTotal, Google Safe Browsing, URLhaus, or a comparable service for reputation and malware signals.
Multi-label classification	Show all matching attack categories instead of only the first primary match.
Scan history	Allow users to review previous checks with timestamps and result summaries.
Downloadable report	Export a scan result as a PDF or shareable security summary.
Backend isolation	Move analysis and URL fetching to a protected backend or sandboxed worker.
Machine-learning model	Use a trained model for feature extraction and classification alongside deterministic rules.
Admin analytics	Track category frequency, false positives, and user feedback for rule improvement.

11 11. Team Details

Role	Name
Team Lead	A.V.V. Satyanarayana
Team Member	N. Sai Sudheer
Team Member	K. Shanmukha

The team is responsible for designing the user experience, implementing the URL analysis workflow, organizing the attack-classification rules, validating the result states, and preparing the project for future threat-intelligence integration.

12 12. Conclusion

The AI-Powered URL Detector provides a focused and understandable way to inspect URLs before clicking them. Its most important contribution is not only the safe or danger color state, but the explanation that follows it. By naming the possible attack, describing the likely purpose, and teaching the user about the possible impact, the project turns URL checking into a practical cyber-security awareness experience.

The current implementation is a strong foundation for an academic demonstration and can be extended with real reputation feeds, isolated backend analysis, multi-label results, and persistent reporting without changing the core user workflow.

13 References

- [1] <https://owasp.org/www-community/attacks/> “OWASP Web Application Security Community — Attacks”
- [2] <https://owasp.org/www-project-top-ten/> “OWASP Top 10 Web Application Security Risks”
- [3] <https://developer.mozilla.org/en-US/docs/Web/HTTP/Redirections> “MDN Web Docs — HTTP redirections”
- [4] <https://www.cisa.gov/stopthinkconnect/phishing> “CISA — Recognize and Report Phishing”